

**Problem # 1:** Multiple choice question. [10 x 1pt]

- 1) Suppose the first 4 numbers in the Fibonacci sequence are given as 1,1,2,3. What is the eighth term of the Fibonacci sequence?  
A. 13                      B. 34                      C. 8                      D. 21
- 2) The worst-case time complexity of ----- is  $O(n^2)$ .  
A. Merge sort      B. Quick sort      C. Binary search      D. Sequential search
- 3) When does the worst case of Quicksort occur?  
A. Array is already sorted in the same order  
B. Array is already sorted in reverse order  
C. All elements of the array are the same  
D. All of the above
- 4) The solution of the recurrence in the table below is:

$$F(n)=F(n-1)+3n, \quad T(1)=5$$

- A.  $3 * n + 2$                       B.  $n * \lg n$   
C.  $1.5 * n^2 + 1.5 * n + 2$                       D. None of the above
- 5) Given the below sorted array:  
 $arr[] = \{1, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59\}$   
To determine if  $x = 37$  exists in the above array using the binary search algorithm, the numbers compared in order with  $x$  are:  
A. 19, 31, 47, 37                      B. 23, 31, 43, 37  
C. 23, 41, 31, 37                      D. None of the above
- 6) Minimum number of moves required to solve a tower of Hanoi problem with  $n$  disks is -----:  
A.  $2^n$                       B.  $n^2$                       C.  $n^2 - 1$                       D.  $2^n - 1$
- 7) If the size of an array is 2048, then the number of comparisons done by binary search and sequential search is ----- respectively.  
A. 12, 2048                      B. 11, 2048                      C. 2048, 12                      D. 2048, 11



8) The big theta of

$$F(n) = 9F\left(\frac{n}{3}\right) + n$$

A.  $n * \log n$

B.  $n^2$

C.  $n$

D.  $\log n$

9) Which of the following is the correct order of growth (from the lowest to highest)?

A.  $\theta(\lg n), \theta(n \lg n), \theta(n^4), \theta(n^n), \theta(3^n), \theta(n!)$

B.  $\theta(\lg n), \theta(n \lg n), \theta(n^4), \theta(3^n), \theta(n!), \theta(n^n)$

C.  $\theta(\lg n), \theta(n \lg n), \theta(n^4), \theta(n!), \theta(3^n), \theta(n^n)$

D. None of the above

10) In mergesort to merge two subarrays  $U = [12, 18, 23, 29]$  and  $V = [14, 17, 22, 31, 33]$  into one array  $S$ , how many times the basic operation (comparison) in merge algorithm that will be executed?

A. 5

B. 6

C. 7

D. None of the above

\*\*\* Write the correct answers of the problem # 1 in the table below:

| <u>Q1</u> | <u>Q2</u> | <u>Q3</u> | <u>Q4</u> | <u>Q5</u> | <u>Q6</u> | <u>Q7</u> | <u>Q8</u> | <u>Q9</u> | <u>Q10</u> |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|------------|
| <b>D</b>  | <b>B</b>  | <b>D</b>  | <b>C</b>  | <b>C</b>  | <b>D</b>  | <b>A</b>  | <b>B</b>  | <b>B</b>  | <b>C</b>   |

**Problem # 2:** Answer the following questions for the given below recurrence equation

$$F(n) = 4F(n-1) - 3F(n-2)$$

1) What is the characteristic equation? [2pts]

**Sol:**

$$r^2 - 4 * r + 3 = 0$$

2) What are the roots of the characteristic equation? [2pts]

**Sol:**

$$(r-3) * (r-1) = 0$$

$$r1 = 3$$

$$r2 = 1$$



3) What is the general form of the solution of the recurrence equation? [2pts]

**Sol:**

$$F(n) = C1 * r1^n + C2 * r2^n$$

$$F(n) = C1 * (3)^n + C2 * (1)^n$$

4) What is the particular form of the solution of the recurrence equation? [2pts]

$$F(0) = 0:$$

$$C1 + C2 = 0$$

$$C1 = -C2$$

$$F(1) = 1$$

$$3C1 + C2 = 1$$

$$-3C2 + C2 = 1$$

$$\text{So: } C2 = -0.5, \quad C1 = 0.5$$

$$F(n) = 0.5 * (3)^n - 0.5 * (1)^n$$

**Problem # 3:** Find the time complexity for the following codes. [3 \* 1pt]

|                                                                                                                                                                     |                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <pre> 1) While ((k++&lt;n) {     for (int i = 0; i &lt; n; ++i)         cout&lt;&lt;i;     for (int j = 0; j &lt; m; ++j)         cout&lt;&lt;j ; } </pre>          | <p>Time complexity:</p> $T(n) = n^2 + n * m$                  |
| <pre> 2) void fun(int n) {     if (n==1)         cout&lt;&lt;"*";     else     {         cout&lt;&lt;"*";         cout&lt;&lt;"*";         fun(n-1);     } } </pre> | <p>Time complexity in recursion form:</p> $F(n) = F(n-1) + 2$ |



|                                                                                                                                                                      |                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| }                                                                                                                                                                    |                                                             |
| <pre> 2) int fun(int a[], int n) {     int max = a[0];     int i;     for (i=1; i&lt;n; i++)         if (a[i]&gt;max)             max=a[i];     return max; } </pre> | <p>Time complexity:</p> <p><b><math>T(n) = n</math></b></p> |

**Problem # 4:** solve the following recurrence using backward substitutions. [5pts]

$$T(n) = 2T\left(\frac{n}{2}\right) + 1, \quad T(1) = 1$$

**Sol:**

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 = 2^1 T\left(\frac{n}{2^1}\right) + (2^1 - 1) \dots \dots \dots (1)$$

$$T(n) = 2 \left[ 2T\left(\frac{\frac{n}{2}}{2}\right) + 1 \right] + 1 = 4T\left(\frac{n}{4}\right) + 3$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 3 = 2^2 T\left(\frac{n}{2^2}\right) + 3 = 2^2 T\left(\frac{n}{2^2}\right) + (2^2 - 1) \dots \dots \dots (2)$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 3 = 4 \left[ 2T\left(\frac{\frac{n}{4}}{2}\right) + 1 \right] + 3 = 8T\left(\frac{n}{8}\right) + 7 = 2^3 T\left(\frac{n}{2^3}\right) + (2^3 - 1) \dots \dots (3)$$

**General form for K iteration is:**

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + (2^k - 1)$$

Assume  $\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \lg n$

$$T(n) = 2^{\lg n} T\left(\frac{n}{2^{\lg n}}\right) + (2^{\lg n} - 1)$$

$2^{\lg n} = n$  so:

$$T(n) = nT\left(\frac{n}{n}\right) + (n - 1)$$

$$T(n) = nT(1) + (n - 1)$$

$$T(n) = 2n - 1$$



**Problem # 5:** If the C++ program below represents the algorithm that find maximum element in the array using divide and conquer. Find the worst-case time complexity of this algorithm. [4pts]

```
int max(int A[], int low, int high) {  
    if (low == high )  
        return A[low];  
    int mid= (low+high)/2;  
    int u = max(A, low, mid);  
    int v = max(A, mid+1, high);  
    return u>v ? u:v;  
}
```

**Sol:**

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$
$$T(1) = 0$$

**Good Luck**